

North East University Bangladesh

**Urban Mobility Demand Forecasting:
Time Series Prediction Using Deep Learning
on the Seoul Bike Sharing Dataset**

Submitted to:

Razorshi Prozzwal Talukder
Lecturer
Department of CSE
North East University Bangladesh

Submitted by:

Rohit Kumar Sen
ID: 0562220005101030
8th Semester
Department of CSE
North East University Bangladesh

Urban Mobility Demand Forecasting Using Deep Learning on Seoul Bike-Sharing Hourly Data

Rohit Kumar Sen

Department of Computer Science and Engineering
North East University Bangladesh

Abstract —

Accurate forecasting of hourly bike-sharing demand supports fleet rebalancing and operational planning in urban mobility systems. This study develops and compares deep learning models on the Seoul Bike Sharing dataset (2017–2018), treating demand prediction as a regression problem over 8,297 cleaned hourly records. A chronological 70/15/15 train–validation–test split and 24-hour input windows ensure evaluation on unseen future time. We implement Vanilla LSTM, Bidirectional LSTM, CNN-LSTM, regime-specific specialists, temporal attention, a 24-step encoder–decoder, and Keras Tuner optimization under a unified protocol. The tuned CNN-LSTM with raw weather features achieves the best one-step performance (RMSE 254.3 bikes, MAE 178.9, R^2 0.781). Raw rainfall, snowfall, and visibility outperform a composite severity index. Multi-step forecasting attains R^2 0.56 across 24 horizons. Results show that hybrid convolutional–recurrent models are well suited to this demand forecasting task.

Keywords — bike sharing, demand forecasting, LSTM, CNN-LSTM, time series, deep learning, Seoul dataset.

1. Introduction

Shared bicycle systems reduce congestion and support short urban trips. For operators, the central operational question is how many bicycles will be needed in the next hour—and, for planning, how demand may evolve over the coming day. Forecasting errors lead to stockouts at stations or inefficient redistribution of idle bikes.

Traditional statistical methods (ARIMA, exponential smoothing) model temporal structure explicitly but struggle with nonlinear interactions between weather, calendar effects, and past demand. Deep learning sequence models can learn such patterns from data when sufficient history and features are available. This project applies modern recurrent and hybrid architectures to hourly Seoul bike-sharing records.

We formulate the problem as **regression**: the target variable is the continuous count of rented bicycles per hour. Performance is measured with RMSE and MAE (in bikes) and coefficient of determination (R^2). We do not use classification accuracy, as the output is not a discrete label.

1.1 Objectives

- Design a reproducible preprocessing and sequence-generation pipeline for hourly time series.
- Compare LSTM, BiLSTM, and CNN-LSTM baselines under identical training settings.
- Evaluate raw weather features versus an engineered weather severity index.

- Assess peak-hour and off-peak specialist models for heterogeneous demand regimes.
- Extend to attention-based and 24-hour encoder–decoder forecasting.
- Optimize the best architecture with automated hyperparameter search.

2. Dataset

The **Seoul Bike Sharing Demand** dataset is a publicly available collection of hourly observations. Each record includes rental count, time of day, meteorological variables, and categorical attributes (season, holiday, system functioning status). The source file contains 8,760 hourly rows spanning 2017–2018.

2.1 Variables

The **target variable** is *Rented Bike Count*. **Predictor variables** include temperature, humidity, wind speed, visibility, solar radiation, rainfall, snowfall, hour, and encoded season/holiday/functioning-day indicators. After feature engineering (lags and rolling statistics), the modeling table contains 8,297 rows.

2.2 Experimental split

Time series data require temporal splitting: the model must never train on future data that would not be available at prediction time. We use a chronological **70% train, 15% validation, 15% test** split without shuffling. Validation guides early stopping; the test set is the final future segment. With a 24-hour lookback window, the test partition yields **1,222** one-step forecast sequences.

3. Preprocessing

Preprocessing transforms raw CSV data into normalized sequence tensors suitable for Keras models. All steps are applied in temporal order; scalers are fit on training data only to prevent information leakage from validation or test periods.

3.1 Cleaning and transformation

Non-functioning days are removed. Extreme rental counts are capped at the 99th percentile to reduce the influence of rare spikes. Rainfall and snowfall receive a $\log(1 + x)$ transform to handle skewness. Dew point temperature is dropped because of high correlation with humidity.

3.2 Feature engineering

Calendar features include day of week, month, and weekend indicator. **Cyclic encoding** represents hour and month as sine–cosine pairs so that 23:00 and 00:00 are close in feature space. **Peak-hour flag** marks commute periods (07:00–09:00 and 17:00–20:00). **Lag features** (1 h, 24 h, 168 h) and **rolling means** (3 h, 24 h) inject recent demand history. A composite *weather_severity* index is defined as $\text{rainfall} + \text{snowfall} + (100 - \text{visibility}/100)$ for ablation experiments.

3.3 Scaling and windowing

Min–Max scaling maps features and target to $[0, 1]$ using training-set statistics only. Each training example is a tensor of shape $(24, F)$ —24 consecutive hours of F features—with label equal to the bike

count at the following hour (one-step) or the next 24 counts (multi-step).

4. Methodology

4.1 Experimental protocol

Experiments use TensorFlow 2.21.0 with random seed 42. Models train for up to 50 epochs with batch size 256, Adam optimizer (learning rate 10^{-3} for baselines), and mean squared error loss. **Early stopping** monitors validation loss (patience 8) and restores best weights. **ReduceLROnPlateau** halves the learning rate when progress stalls. Best checkpoints are saved under *artifacts/*. All reported metrics are computed after inverse scaling to original bike counts.

4.2 Feature ablations

Two parallel feature configurations isolate weather representation: (i) **raw_weather** retains rainfall, snowfall, and visibility; (ii) **severity_index** replaces them with the single severity score. All other features and the temporal split remain fixed.

4.3 Deep learning models

This section describes each architecture in general terms, then states how it is configured in our notebook. All one-step models share the same 24-hour input unless noted.

4.3.1 Long Short-Term Memory (LSTM)

The **Long Short-Term Memory (LSTM)** network is a recurrent neural network designed to model sequential data. Unlike standard RNNs, which often fail to learn long-range patterns due to vanishing gradients, an LSTM cell uses gating mechanisms—input, forget, and output gates—to decide what information to retain, update, or discard across time steps. This makes LSTMs effective for time series where both recent and moderately distant past values influence the future.

Implementation in this project. We use a single LSTM layer with 64 hidden units and dropout rate 0.2, followed by a dense layer with one output neuron. The network reads the full 24-hour window and produces the next-hour bike count.

Role in the experiment. Serves as the simplest deep learning baseline for comparison with more complex architectures.

4.3.2 Bidirectional LSTM (BiLSTM)

A **Bidirectional LSTM** runs two LSTM layers on the same input sequence: one processes time forward, the other backward. Their outputs are combined (typically concatenated) so that each position can depend on both earlier and later observations within the window. BiLSTMs often improve performance on sequence labeling tasks where full context is available.

Implementation in this project. We wrap an LSTM(64) in a Bidirectional layer with dropout 0.2, then apply Dense(1). For a 24-hour history, the model can use information from hour 23 when interpreting hour 1 within the window.

Role in the experiment. Tests whether bidirectional context within the lookback window improves demand forecasts. Note: bidirectional processing uses later timesteps inside the window;

strict real-time streaming may prefer unidirectional models.

4.3.3 Convolutional LSTM (CNN-LSTM)

The **CNN-LSTM** hybrid combines one-dimensional convolution with recurrent modeling. A **Conv1D** layer scans the time axis with small filters (kernel size 3), detecting local patterns such as sudden weather changes or rapid demand shifts. **Causal padding** ensures each time position depends only on current and past inputs—no lookahead within the window. Max pooling reduces temporal resolution; the LSTM then aggregates the convolved features into a single representation for prediction.

Implementation in this project. Architecture: Conv1D(64 filters, kernel 3, causal) → ReLU → MaxPool1D(2) → Dropout(0.2) → LSTM(64) → Dense(1).

Role in the experiment. Our strongest baseline family; captures both short local motifs and longer temporal structure.

4.3.4 Peak and off-peak specialist models

Bike demand is **heterogeneous across time of day**: morning and evening peaks reflect commuting, while off-peak hours show smoother, weather-driven variation. A single global model must compromise between these regimes. **Specialist models** train separate networks on subsets of data where the target hour falls in peak or off-peak periods.

Implementation in this project. We reuse the CNN-LSTM architecture but filter training, validation, and test sequences by the hour being predicted (peak: 7–9 and 17–20). Masks align with the label hour, not the start of the input window.

Role in the experiment. Evaluates whether regime-specific models reduce error on peak (n=357) and off-peak (n=865) test subsets.

4.3.5 CNN-LSTM with temporal attention

Attention mechanisms allow a neural network to focus on the most informative time steps when making a prediction. After the LSTM produces a hidden vector at each (downsampled) time position, a small feed-forward network scores each position. **Softmax** converts scores to weights summing to 1; the weighted sum of LSTM outputs forms the context vector for the final dense prediction. Attention is widely used in sequence-to-sequence models and can aid interpretability by revealing which past hours received high weight.

Implementation in this project. Conv1D and pooling match the CNN-LSTM stack; LSTM uses `return_sequences=True`; additive attention ($\tanh \rightarrow \text{Dense}(1) \rightarrow \text{Softmax}$) produces context → Dense(1).

Role in the experiment. Tests whether explicit temporal weighting improves over the simpler pooling-plus-LSTM design.

4.3.6 Encoder–decoder LSTM (24-step forecasting)

An **encoder–decoder** architecture maps a variable-length input sequence to a variable-length output sequence. The **encoder** LSTM reads the 24-hour input and compresses it into a fixed-size state vector (hidden and cell states). The **decoder** LSTM then generates outputs for each future time step. Here we forecast all 24 hours ahead in one forward pass, which is harder than single-step prediction

because errors can compound and long-range structure must be inferred.

Implementation in this project. Encoder LSTM(64) with `return_state=True`; `RepeatVector(24)` tiles the encoder output; decoder LSTM(64, `return_sequences=True`) initialized from encoder states; `TimeDistributed(Dense(1))` predicts one bike count per future hour.

Role in the experiment. Provides day-ahead demand trajectories for operational planning; evaluated with per-horizon RMSE.

4.3.7 Hyperparameter tuning (Keras Tuner)

Hyperparameter tuning automates the search for architectural and optimization settings that minimize validation loss. Manual tuning is time-consuming when many combinations exist (filter counts, kernel sizes, LSTM width, dropout, learning rate). **Random search** samples configurations from a defined space and trains each for a limited budget; it often finds strong models more efficiently than exhaustive grid search in high dimensions.

Implementation in this project. Keras Tuner `RandomSearch` with `max_trials=10` optimizes `val_loss`. Search space: `conv_filters {32,64,96}`, `kernel_size {2,3,5}`, `lstm_units {32,64,96}`, `dropout 0.1–0.5`, `lr {1e-3, 5e-4, 1e-4}`. Best trial fine-tuned 50 epochs. Best result: 64 filters, $k=3$, 96 LSTM units, `dropout 0.2`, $lr=5 \times 10^{-4}$.

Role in the experiment. Produces `cnn_lstm_tuned_raw_weather`, the recommended one-step model (RMSE 254.3 on test).

4.4 Evaluation metrics

RMSE (Root Mean Squared Error) reports typical error in bikes per hour, penalizing large mistakes more heavily. **MAE** (Mean Absolute Error) is the average absolute deviation in bikes. **R²** measures the proportion of variance in demand explained relative to predicting the mean. **sMAPE** (symmetric mean absolute percentage error) provides a scale-free comparison that is more stable than MAPE when counts are low.

5. Results

Tables report performance on inverse-scaled predictions (original bike units).

5.1 One-step forecasting (test set)

Model	Split	n	RMSE	MAE	R ²	sMAPE%
Off-peak specialist	off-peak	865	232.1	177.2	0.704	34.1
CNN-LSTM (tuned)	test	1222	254.3	178.9	0.781	28.6
CNN-LSTM (baseline)	test	1222	264.0	190.1	0.764	30.5
CNN-LSTM + attention	test	1222	286.6	207.8	0.722	33.0
BiLSTM	test	1222	294.1	202.4	0.707	30.5
Vanilla LSTM	test	1222	303.0	215.3	0.689	32.3
CNN-LSTM (severity only)	test	1222	306.4	216.5	0.682	32.6

Model	Split	n	RMSE	MAE	R ²	sMAPE%
Peak specialist	peak	357	337.1	249.9	0.668	24.8

Table I. One-step test results (raw weather unless noted).

The tuned CNN-LSTM achieves the best full-test RMSE (254.3), improving 3.7% over the hand-tuned CNN-LSTM baseline. On average, predictions deviate by approximately 179 bikes per hour while explaining 78.1% of demand variance. The off-peak specialist attains lower RMSE (232.1) but only on off-peak test hours; it is not directly comparable to full-test rankings.

5.2 Validation and feature ablation

Model	RMSE	MAE	R ²
CNN-LSTM (raw)	290.6	215.9	0.801
CNN-LSTM (severity)	312.9	232.2	0.769
Vanilla LSTM (raw)	324.8	241.7	0.751

Table II. Validation-set comparison ($n = 1,220$).

For CNN-LSTM, raw weather channels outperform the severity index by roughly 42 bikes RMSE on test (264.0 vs 306.4). The architecture ladder on raw features is: CNN-LSTM < BiLSTM < Vanilla LSTM. Attention (RMSE 286.6) did not surpass plain CNN-LSTM (264.0). Peak-hour RMSE (337.1) exceeds off-peak (232.1) by about 105 bikes, confirming greater difficulty during commute periods.

5.3 Multi-step (24-hour) results

The encoder–decoder model yields overall RMSE 359.7, MAE 250.1, and R² 0.562 when all horizons are pooled. Horizon-1 RMSE is 336.0; horizon-24 RMSE is 370.7 (+10.3%). Error increases through hour 12 (+11.2%) and then stabilizes through hour 24 (+0.3%), indicating that the model captures a stable daily profile but not fine-grained hour-ahead detail.

h	1	2	3	4	5	6	7	8	9	10	11	12
RMSE	336	346	347	347	348	348	348	350	355	365	372	374
h	13	14	15	16	17	18	19	20	21	22	23	24
RMSE	371	368	366	365	365	364	364	364	365	365	365	371

Table III. RMSE by forecast horizon (hours ahead).

6. Conclusion

This project implemented a complete deep learning workflow for urban bike demand forecasting on Seoul hourly data, from preprocessing through multi-architecture comparison and automated tuning. Chronological splitting, train-only scaling, and inverse-transformed metrics ensure that results reflect realistic generalization to future time.

The **tuned CNN-LSTM with raw weather features** delivered the best one-step performance (RMSE 254.3, MAE 178.9, R² 0.781). Combining convolutional feature extraction with LSTM-based

temporal integration proved more effective than LSTM or BiLSTM alone. Raw meteorological channels carried more information than a single severity index for the hybrid model. Specialist and multi-step models further characterized peak versus off-peak difficulty and day-ahead forecast behavior. These findings demonstrate that deep sequence models can support data-driven bike-sharing operations when paired with careful temporal methodology.